



Girls' Programming Network

Candy Catcher!

FOR TUTOR EYES ONLY

This project was created by GPN Australia for GPN sites all around Australia!

This workbook and related materials were created by tutors at:

Sydney and Melbourne



Girls' Programming Network

If you see any of the following tutors don't forget to thank them!!

Writers

Isabella Hogan

Angela Sojan

Sophie Lee-Goh

Testers

Amanda Hogan

Scaffold

```
# <insert your name here>
# imports

# create constants

# create parent class
# create child classes

# make hero
# set grid size
# create a list of game objects
# add 5 candies to the game
# add 3 bombs to the game

# manage game state
# read high score from file and set target

# print welcome -- update when high score is added

# manage key controls
    # down up left right

# update everything
    # stop updating if game is over
    # Check collisions with all game objects (demonstrates
polymorphism)
        # behaviour depends on object type
        # Prevent negative scores

    # Check win condition
    # update timer

# draw everything to screen
    # clear screen

    # draw all game objects (demonstrates polymorphism)
    # draw hero
    # draw text
    # update high score file if new high score achieved

# runs everything
```

Part 1: Welcome to “Candy Catcher”

1.1: Who am I?

The code should look like this (no bonuses):

```
...  
# My name is ____  
# imports  
...
```

1.2: Welcome to ‘Candy Catcher’

The code should look like this (no bonuses):

```
...  
# print welcome -- update when high score is added  
print("""-----  
Welcome to Candy Catcher!  
-----  
Collect candies (1 point each)  
Avoid bombs (3 seconds off timer each)  
Reach 20 points to win!  
Press arrow keys to move.  
-----""")  
...
```

1.3: Setup Pygame Zero

The code should look like this (no bonuses):

```
...  
# imports  
import pgzrun  
...  
# runs everything  
pgzrun.go()  
...
```

1.4: Make a screen

The code should look like this (no bonuses):

```
...
# create constants
WIDTH = 640
HEIGHT = 480
...
```

Full solution after Part 1

The code should look like this (no bonuses):

```
# My name is ____
# imports
import pgzrun

# create constants
WIDTH = 640
HEIGHT = 480

# create parent class

# create child classes

# make hero

# set grid size

# create a list of game objects

# add 5 candies to the game

# add 3 bombs to the game

# manage game state

# read high score from file and set target

# print welcome -- update when high score is added
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
""")
```

Press arrow keys to move.

-----"")

manage key controls

 # up down left right with boundary checks

update everything

 # stop updating if game is over

 # Check collisions with all game objects (demonstrates
polymorphism)

 # behavior depends on object type

 # Prevent negative scores

 # Check win condition

 # update timer

draw everything to screen

 # clear screen

 # draw all game objects (demonstrates polymorphism)

 # draw hero

 # draw text

 # update high score file if new high score achieved

runs everything

pgzrun.go()

Part 2: Creating the hero

Tutor hint: the hero will be a work in progress for the whole of part 2. It is expected that it will not likely behave in the way that students would have initially expected (assuming that they have not worked with Pygame before).

2.1: Creating the hero

The code should look like this (no bonuses):

```
...
# make hero
hero = Actor("hero")
...
```

2.2: Setting up the draw function

The code should look like this (no bonuses):

```
...
# draw everything to screen
def draw():
    ...
```

2.3: Drawing the hero to the screen

The code should look like this (no bonuses):

```
...
# draw everything to screen
def draw():
    # clear screen

    # draw all game objects (demonstrates polymorphism)

    # draw hero
    hero.draw()
    ...
```

2.4: Keeping the hero on the screen

The code should look like this (no bonuses):

```
...
# update everything
def update():
    pass
...
```

Full solution after Part 2

The code should look like this (no bonuses):

```
# <insert your name here>
# imports
import pgzrun

# create constants
WIDTH = 640
HEIGHT = 480

# create parent class

# create child classes

# make hero
hero = Actor("hero")

# set grid size

# create a list of game objects

# add 5 candies to the game

# add 3 bombs to the game

# manage game state

# read high score from file and set target

# print welcome -- update when high score is added
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
""")
```

```

Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----"")
# manage key controls
    # up down left right with boundary checks

# update everything
def update():
    pass
    # stop updating if game is over

    # Check collisions with all game objects (demonstrates
polymorphism)
        # behavior depends on object type

        # Prevent negative scores

    # Check win condition

    # update timer

# draw everything to screen
def draw():
    # clear screen

    # draw all game objects (demonstrates polymorphism)

    # draw hero
    hero.draw()

    # draw text

    # update high score file if new high score achieved

# runs everything
pgzrun.go()

```

Part 3: I like to move it move it!

3.1: Adding the maths

The code should look like this (no bonuses):

```
...
# set grid size
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square
...
```

3.2: Moving the hero down

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):
    # down up left right
    hero.bottom = hero.bottom + square
...
```

3.3: Moving down when we want to

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):
    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
...
```

3.4: Adding other moves

The code should look like this (no bonuses):

```
...
# manage key controls
def on_key_down(key):
    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square
...
```

3.5: Painting it black

The code should look like this (no bonuses):

```
...
# draw everything to screen
def draw():
    # clear screen
    screen.fill("black")
...
```

Full solution after Part 3

The code should look like this (no bonuses):

```
# <insert your name here>
# imports
import pgzrun

# create constants
WIDTH = 640
HEIGHT = 480

# create parent class
```

```

# create child classes

# make hero
hero = Actor("hero")

# set grid size
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# create a list of game objects

# add 5 candies to the game

# add 3 bombs to the game

# manage game state

# read high score from file and set target

# print welcome -- update when high score is added
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# manage key controls
def on_key_down(key):
    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square

# update everything
def update():
    pass
    # stop updating if game is over

```

```

    # Check collisions with all game objects (demonstrates
    polymorphism)
        # behavior depends on object type

        # Prevent negative scores

    # Check win condition

    # update timer

# draw everything to screen
def draw():
    # clear screen
    screen.fill("black")

    # draw all game objects (demonstrates polymorphism)

    # draw hero
    hero.draw()

    # draw text

    # update high score file if new high score achieved

# runs everything
pgzrun.go()

```

Bonus 3.6: Keeping the hero on screen

The code should look like this:

```

...
# down up left right
    if key == keys.DOWN:
        hero.bottom = min(hero.bottom + square, HEIGHT)
    if key == keys.UP:
        hero.top = max(hero.top - square, 0)
    if key == keys.LEFT:
        hero.left = max(hero.left - square, 0)
    if key == keys.RIGHT:
        hero.right = min(hero.right + square, WIDTH)
...

```

Part 4: A class of one's own

4.1: Making the game_object

The code should look like this (no bonuses):

```
...
# create parent class
class game_object:
    def __init__(self,filename):
...

```

4.2: Making the candy

The code should look like this (no bonuses):

```
...
# create parent class
class game_object:
    def __init__(self,filename):
        self.actor = Actor(filename)
        self.actor.top = square
        self.actor.right = square
...
# create a list of game objects
candy = game_object("candy")
...
    # draw all game objects (demonstrates polymorphism)
    candy.actor.draw()
...

```

4.3: Random position

The code should look like this (no bonuses):

```
...
# imports
import random
...
# create parent class
class game_object:
    def __init__(self,filename):
        self.actor = Actor(filename)
        self.set_position()

    def set_position(self):
        self.actor.top = square * random.randint(0,num_squares_height-1)
        self.actor.right = square * random.randint(0,num_squares_width-1)
...
```

4.4: Collecting the candy

The code should look like this (no bonuses):

```
...
# create parent class
class game_object:
    def __init__(self,filename):
        self.actor = Actor(filename)
        self.set_position()

    def set_position(self):
        self.actor.top = square * random.randint(0,num_squares_height-1)
        self.actor.right = square * random.randint(0,num_squares_width-1)

    def on_collision(self):
        self.set_position()
...
# Check collisions with all game objects (demonstrates polymorphism)
if hero.colliderect(candy.actor):
    candy.on_collision()
...
```

4.5: Many candies

The code should look like this (no bonuses):

```
...
# create a list of game objects
game_objects = [game_object("candy"),game_object("candy"),game_object("candy")]
...
    # Check collisions with all game objects (demonstrates polymorphism)
    for i in game_objects:
        if hero.colliderect(i.actor):
            i.on_collision()
...
    # draw all game objects (demonstrates polymorphism)
    for i in game_objects:
        i.actor.draw()
...
```

Full solution after Part 4

The code should look like this (no bonuses):

```
# <insert your name here>
# imports
import pgzrun
import random

# create constants
WIDTH = 640
HEIGHT = 480

# create parent class
class game_object:
    def __init__(self,filename):
        self.actor = Actor(filename)
        self.set_position()

    def set_position(self):
        self.actor.top = square * random.randint(0,num_squares_height-1)
        self.actor.right = square * random.randint(0,num_squares_width-1)

    def on_collision(self):
        self.set_position()

# create child classes

# make hero
hero = Actor("hero")

# set grid size
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square
```

```

# create a list of game objects
game_objects = [game_object("candy"), game_object("candy"), game_object("candy")]

# add 5 candies to the game

# add 3 bombs to the game

# manage game state

# read high score from file and set target

# print welcome -- update when high score is added
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# manage key controls
def on_key_down(key):
    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:
        hero.right = hero.right + square

# update everything
def update():
    pass
    # stop updating if game is over

    # Check collisions with all game objects (demonstrates polymorphism)
    for i in game_objects:
        if hero.colliderect(i.actor):
            i.on_collision()
            # behavior depends on object type

            # Prevent negative scores

    # Check win condition

    # update timer

# draw everything to screen
def draw():
    # clear screen
    screen.fill("black")

    # draw all game objects (demonstrates polymorphism)
    for i in game_objects:
        i.actor.draw()

```

```
# draw hero
hero.draw()

# draw text

# update high score file if new high score achieved

# runs everything
pgzrun.go()
```

Bonus 4.6: Unique placement

The code should look like this:

```
...
def set_position(self):
    global cur_positions
    old_top = self.actor.top
    old_right = self.actor.right

    new_top = square * random.randint(0,num_squares_height-1)
    new_right = square * random.randint(0,num_squares_width-1)
    while ((new_top,new_right)==(old_top,old_right)) or (new_top,new_right)
in cur_positions:
        new_top = square * random.randint(0,num_squares_height-1)
        new_right = square * random.randint(0,num_squares_width-1)

    if (old_top,old_right) in cur_positions:
        cur_positions.remove((old_top,old_right))

    self.actor.top = new_top
    self.actor.right = new_right
    cur_positions.add((new_top,new_right))
...
```

Part 5: So many objects

5.1: Making the candy and hazards

The code should look like this (no bonuses):

```
...
# create child classes
class candy(game_object):
    def __init__(self,filename):
        super().__init__(filename)

class bomb(game_object):
    def __init__(self,filename):
        super().__init__(filename)
...
# create a list of game objects
game_objects = []

# add 5 candies to the game
for i in range(5):
    game_objects.append(candy("candy"))
# add 3 bombs to the game
for i in range(3):
    game_objects.append(bomb("bomb"))
...
```

5.2: Score and health

The code should look like this (no bonuses):

```
...
    def on_collision(self):
        print("this should be handled by the subclasses")
...
# create child classes
class candy(game_object):
    def __init__(self,filename):
        super().__init__(filename)

    def on_collision(self):
        self.set_position()
        return "score",1

class bomb(game_object):
    def __init__(self,filename):
        super().__init__(filename)

    def on_collision(self):
        self.set_position()
        return "health",1

...
# manage game state
score = 0
health = 5
...
```

5.3: Score and health part 2

The code should look like this (no bonuses):

```
...
# update everything
def update():
    global score, health
    # pass
    # stop updating if game is over

    # Check collisions with all game objects (demonstrates
    polymorphism)
    for i in game_objects:
        if hero.colliderect(i.actor):
            change, value = i.on_collision()
            # behavior depends on object type
            if change=="score":
                score+=value
            else:
                health-=value
            print(f"score:{score} health:{health}")
    ...
    # draw text
    screen.draw.text(f'Score: {score}', (10, 10),
    color="white", fontsize=30)
    screen.draw.text(f'Hearts: {int(health)}/5', (10, 25),
    color="white", fontsize=30)
    ...
```

5.4: Game over

The code should look like this (no bonuses):

```
...
def update():
    global score,health,gameover
    # pass
    # stop updating if game is over
    if gameover==True:
        return
...
        # Prevent negative scores
        if health==0:
            gameover=True
...
# manage game state
score = 0
health = 5
gameover = False
...
```

Full solution after Part 5

The code should look like this (no bonuses):

```
# <insert your name here>
# imports
import pgzrun
import random

# create constants
WIDTH = 640
HEIGHT = 480

# create parent class
class game_object:
    def __init__(self,filename):
        self.actor = Actor(filename)
        self.set_position()

    def set_position(self):
        self.actor.top = square * random.randint(0,num_squares_height-1)
        self.actor.right = square * random.randint(0,num_squares_width-1)

    def on_collision(self):
        print("this should be handled by the subclasses")

# create child classes
class candy(game_object):
```

```

        def __init__(self,filename):
            super().__init__(filename)

        def on_collision(self):
            self.set_position()
            return "score",1

class bomb(game_object):
    def __init__(self,filename):
        super().__init__(filename)

        def on_collision(self):
            self.set_position()
            return "health",1

# make hero
hero = Actor("hero")

# set grid size
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# create a list of game objects
game_objects = []

# add 5 candies to the game
for i in range(5):
    game_objects.append(candy("candy"))
# add 3 bombs to the game
for i in range(3):
    game_objects.append(bomb("bomb"))

# manage game state
score = 0
health = 5
gameover = False

# read high score from file and set target

# print welcome -- update when high score is added
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# manage key controls
def on_key_down(key):
    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:

```

```

        hero.right = hero.right + square

# update everything
def update():
    global score,health,gameover
    # pass
    # stop updating if game is over
    if gameover==True:
        return

    # Check collisions with all game objects (demonstrates polymorphism)
    for i in game_objects:
        if hero.colliderect(i.actor):
            i.on_collision()
            # behavior depends on object type

            # Prevent negative scores
            if health==0:
                gameover=True

    # Check win condition

    # update timer

# draw everything to screen
def draw():
    # clear screen
    screen.fill("black")

    # draw all game objects (demonstrates polymorphism)
    for i in game_objects:
        i.actor.draw()
    # draw hero
    hero.draw()

    # draw text
    screen.draw.text(f'Score: {score}', (10, 10), color="white", fontsize=30)
    screen.draw.text(f'Hearts: {int(health)}/5', (10, 25), color="white",
    fontsize=30)
    # update high score file if new high score achieved

# runs everything
pgzrun.go()

```

Bonus 5.5: Gameover screen

The code could look like this:

```

...
    if score == 20:
        screen.fill("green")
        screen.draw.text("You have won")
...

```

Part 6: What's the time Mr Wolf?

6.1: Making a timer

The code should look like this (no bonuses):

```
...
# manage game state
score = 0
health = 5
gameover = False
timer = 3600
...
def update():
    global score, health, gameover, timer
    ...
# update timer
    timer -= 1
    if timer == 0:
        gameover = True
        return
...
# draw text
    screen.draw.text(f'Score: {score}', (10, 10),
color="white", fontsize=30)
    screen.draw.text(f'Hearts: {int(health)}/5', (10, 25),
color="white", fontsize=30)
    screen.draw.text(f'Time: {int((timer+59)/60)}', (WIDTH-95,
10), color="white", fontsize=30)
...
```

6.1: Making a timer

The code should look like this (no bonuses):

```
...
# add 5 candies to the game
game_objects.append(candy("candy"))
...
    # update timer
    timer-=1
    if timer==0:
        gameover=True
        return
    if timer%540==0:
        game_objects.append(candy("candy"))
    if timer%900==0:
        game_objects.append(bomb("bomb"))
...
```

Full solution after Part 6

The code should look like this (no bonuses):

```
# <insert your name here>
# imports
import pgzrun
import random

# create constants
WIDTH = 640
HEIGHT = 480

# create parent class
class game_object:
    def __init__(self,filename):
        self.actor = Actor(filename)
        self.set_position()

    def set_position(self):
        self.actor.top = square * random.randint(0,num_squares_height-1)
        self.actor.right = square * random.randint(0,num_squares_width-1)

    def on_collision(self):
        print("this should be handled by the subclasses")

# create child classes
class candy(game_object):
    def __init__(self,filename):
        super().__init__(filename)

    def on_collision(self):
```

```

        self.set_position()
        return "score",1

class bomb(game_object):
    def __init__(self,filename):
        super().__init__(filename)

    def on_collision(self):
        self.set_position()
        return "health",1

# make hero
hero = Actor("hero")

# set grid size
square = hero.width
num_squares_width = WIDTH // square
num_squares_height = HEIGHT // square

# create a list of game objects
game_objects = []

# add 5 candies to the game
game_objects.append(candy("candy"))
# add 3 bombs to the game

# manage game state
score = 0
health = 5
gameover = False
timer = 3600

# read high score from file and set target

# print welcome -- update when high score is added
print("""-----
Welcome to Candy Catcher!
-----
Collect candies (1 point each)
Avoid bombs (3 seconds off timer each)
Reach 20 points to win!
Press arrow keys to move.
-----""")

# manage key controls
def on_key_down(key):
    # down up left right
    if key == keys.DOWN:
        hero.bottom = hero.bottom + square
    if key == keys.UP:
        hero.top = hero.top - square
    if key == keys.LEFT:
        hero.left = hero.left - square
    if key == keys.RIGHT:

# update everything
def update():
    global score,health,gameover,timer
    # pass
    # stop updating if game is over

```

```

if gameover==True:
    return

# Check collisions with all game objects (demonstrates polymorphism)
for i in game_objects:
    if hero.colliderect(i.actor):
        i.on_collision()
        # behavior depends on object type

    # Prevent negative scores
    if health==0:
        gameover=True

# Check win condition

# update timer
timer-=1
if timer==0:
    gameover=True
    return
if timer%540==0:
    game_objects.append(candy("candy"))
if timer%900==0:
    game_objects.append(bomb("bomb"))

# draw everything to screen
def draw():
    # clear screen
    screen.fill("black")

    # draw all game objects (demonstrates polymorphism)
    for i in game_objects:
        i.actor.draw()
    # draw hero
    hero.draw()

    # draw text
    screen.draw.text(f'Score: {score}', (10, 10), color="white", fontsize=30)
    screen.draw.text(f'Hearts: {int(health)}/5', (10, 25), color="white",
    fontsize=30)
    screen.draw.text(f'Time: {int((timer+59)/60)}', (WIDTH-95, 10),
    color="white", fontsize=30)
    # update high score file if new high score achieved

# runs everything
pgzrun.go()

```